

Tema 2

Proiectarea centrata pe sarcini si prototipizarea

Proiect: Kubexplain - Asistent AI pentru Diagnosticarea
Problemelor in Clustere Kubernetes

Axinescu Valentin-Daniel

Cuprins

1. Descrierea temei	2
2. Ce contine portofoliul	2
Sectiunea 1 - Sarcini si cerinte	3
Sectiunea 2 - Primul prototip si parcurgerea lui	3
3. Etapele rezolvarii temei	3
4. Sectiunea 1: Sarcini si cerinte - Kubexplain	4
Introducere: cadrul general al sistemului	4
Introducere: utilizatori potentiali	5
Introducere: contexte de lucru	5
Introducere: la ce va fi folosit sistemul	6
Introducere: constrangerile sistemului	6
Exemple de sarcini	7
Lista de cerinte	10
5. Sectiunea 2: Prototip si parcurgere orientata pe sarcini	12
Parcurgerea sarcinii 1	12
Parcurgerea sarcinii 2	14
Preocupari majore din parcurgerea sarcinilor	15
6. Bibliografie	16

1. Descrierea temei

Prim pas in proiectarea iterativa a unui sistem: proiectarea centrata pe sarcini. Se incepe prin identificarea utilizatorilor si a sarcinilor pe care le au de indeplinit, precum si contextul desfasurarii actiunilor lor. Un sistem este proiectat asa cum trebuie numai daca ia in considerare oamenii, sarcinile si nevoile lor.

Tema propusa - inceperea unei proiectari iterative pentru sistemul Kubexplain, un asistent AI pentru diagnosticarea problemelor in clustere Kubernetes, folosind metoda proiectarii centrate pe sarcini si metode de prototipizare.

Scop - capatarea de experienta in:

- Realizarea unei descrieri bune a sarcinilor utilizatorilor sistemului Kubexplain
- Folosirea descrierii sarcinilor pentru a decide cu privire la cerintele sistemului
- Realizarea unor prototipuri pe baza celor de mai sus
- Evaluarea prototipurilor prin parcurgerea orientata pe sarcini

2. Ce contine portofoliul

- O lista care descrie potentialii utilizatori si contextele in care ei lucreaza
- O lista a sarcinilor reprezentative pe care oamenii se asteapta sa fie facute
- O lista de prioritati pe cerintele sistemului
- Un prototip functional al interfetei
- O parcurgere a prototipului centrata pe sarcini

Sectiunea 1 - Sarcini si cerinte (aprox. 10 pag)

1. Introducere. Descrierea, in termeni generali, a comportamentului sistemului: utilizatori potentiali, contextele lor de lucru, ce fel de sistem se asteapta sa foloseasca.
2. Exemple de sarcini. 5-7 exemple concrete de sarcini cu proprietatile din specificatii.
3. Lista de cerinte. Pe baza exemplurilor de sarcini se extrag principalele cerinte de sistem si se introduc prioritati.

Sectiunea 2 - Primul prototip si parcurgerea lui

1. Prototip - dezvoltarea prototipului care sa satisfaca cerintele principale.
2. Discutii si parcurgere. Parcurgerea prototipului pe baza sarcinilor identificate, evaluand cat de bine interfata se potriveste cu viziunea utilizatorilor.
3. Preocupari majore identificate in urma parcurgerii.

3. Etapele rezolvarii temei

Etapa 1. Generarea unei liste de utilizatori potentiali si a unei liste initiale de sarcini

S-au identificat utilizatorii potentiali ai sistemului Kubexplain prin analiza directa a fluxurilor de lucru DevOps si prin discutii cu colegi care administreaza clustere Kubernetes. S-au observat sarcinile curente pe care le realizeaza: diagnosticarea podurilor cu probleme, verificarea starii nodurilor, analiza logurilor si evenimentelor. Acestea au stat la baza listei initiale de sarcini.

Etapa 2. Validarea sarcinilor

S-a verificat lista sarcinilor cu potentiali utilizatori (ingineri DevOps, administratori de sisteme, studenti cu experienta in Kubernetes).

S-au adaugat sarcini suplimentare precum gestionarea istoricului conversatiilor si atasarea de fisiere de context.

Etapa 3. Deciderea utilizatorilor cheie si a unei prime liste de cerinte

Din exemplele de sarcini si discutiile cu utilizatorii, s-au decis cerintele majore ale sistemului si s-au prioritzat in: a) cele care trebuie incluse neaparat; b) cele care trebuie incluse; c) cele care pot fi incluse; d) cele care se exclud.

Etapa 4. Dezvoltarea prototipului

S-a dezvoltat un prototip functional al aplicatiei web Kubexplain, cu interfata chat, scanere pentru poduri si noduri, sistem de autentificare, istoric conversatii si integrare cu un LLM (Ollama/Llama 3.1) pentru generarea raspunsurilor.

Etapa 5. Parcurgere orientata pe sarcini

S-a testat prototipul prin parcurgerea orientata pe sarcini, identificandu-se probleme potientiale de utilizare si oportunitati de imbunatatire.

4. Sectiunea 1: Sarcini si cerinte - Kubexplain

Introducere: cadrul general al sistemului

Kubexplain este o aplicatie web de tip asistent inteligent, conceputa pentru a ajuta inginerii DevOps si administratorii de sisteme sa diagnosticheze si sa rezolve probleme in clustere Kubernetes. Sistemul combina o interfata de chat conversational cu un model de limbaj (LLM) si mecanisme de colectare automata a datelor din cluster.

- Contextul actual: Diagnosticarea problemelor in Kubernetes presupune utilizarea manuala a comenzilor kubectl (describe, logs, events, get -o json), interpretarea output-urilor complexe si corelarea informatiilor din surse multiple. Acest proces este consumator de timp si necesita experienta semnificativa.
- Problema pe care o rezolva: Kubexplain automatizeaza colectarea datelor diagnostice, le analizeaza cu ajutorul unui LLM si ofera solutii contextuale bazate pe documentatia oficiala Kubernetes si pe experienta acumulata.
- Arhitectura: Sistemul este compus din trei componente principale containerizate cu Docker: un frontend web (Node.js/Express), un backend API (Spring Boot/Java), si un server AI (Spring Boot/Java cu integrare Ollama). Baza de date PostgreSQL stocheaza utilizatorii, conversatiile si datele de context.

Introducere: utilizatori potentiali

- Inginer DevOps cu experienta (utilizator principal): Persoana care administreaza clustere Kubernetes zilnic, cu experienta in kubectl si concepte Kubernetes. Foloseste Kubexplain pentru a accelera diagnosticarea si pentru a obtine sugestii pentru probleme mai putin intalnite. Nu are nevoie de instruire formala, fiind familiarizat cu terminologia.
- Administrator de sisteme/SRE (Site Reliability Engineer): Responsabil cu mentinerea disponibilitatii serviciilor. Utilizeaza Kubexplain in situatii de urgenta (incidente) pentru a identifica rapid cauza unei defectiuni. Are experienta medie spre avansata in Kubernetes.
- Dezvoltator software cu cunostinte de baza in Kubernetes: Persoana care deployeaza aplicatii pe cluster dar nu administreaza infrastructura. Foloseste Kubexplain cand aplicatia sa nu porneasca sau nu functioneaza corect. Necesita explicatii mai detaliate si ghidare pas cu pas.
- Student/practicant in domeniul cloud: Persoana aflata in procesul de invatare a Kubernetes. Foloseste Kubexplain ca instrument educational, punand intrebari si primind explicatii despre concepte si comenzi.

Introducere: contexte de lucru

- Munca de rutina: Verificarea periodica a starii clusterului - scanarea podurilor si nodurilor, identificarea celor cu probleme (CrashLoopBackOff, Pending, ImagePullBackOff). Aceasta sarcina se realizeaza zilnic, de obicei la inceputul zilei de lucru.
- Raspuns la incidente: Cand un serviciu cade sau un pod refuza sa porneasca, inginerul trebuie sa actioneze rapid. Kubexplain este folosit ca prim instrument de investigatie, oferind o analiza automata a logurilor si evenimentelor.
- Invatare si explorare: Studentii si dezvoltatorii folosesc Kubexplain pentru a intelege mai bine conceptele Kubernetes, punand intrebari in limbaj natural si primind explicatii contextualizate.
- Lucrul cu clustere remote: Sistemul este deployat pe un server cloud (OpenStack) si accesat prin browser, permitand diagnosticarea de la distanta fara acces direct la linia de comanda a clusterului.

Introducere: la ce va fi folosit sistemul

Sistemul va gestiona urmatoarele activitati:

- Chat conversational cu AI pentru diagnosticarea problemelor Kubernetes

- o Utilizatorul descrie problema in limbaj natural
- o AI-ul analizeaza si ofera solutii bazate pe documentatia oficiala
- o Suport pentru conversatii multi-tura cu pastrarea contextului
- Scanarea si inspectia resurselor din cluster
 - o Scanare poduri pe namespace cu vizualizare stare (Running, Pending, Failed)
 - o Scanare noduri cu detalii despre rol (Master/Worker) si resurse
 - o Vizualizare detaliata: Describe, JSON, Events, Logs pentru fiecare resursa
- Atasarea de context diagnostic la conversatie
 - o Adaugarea automata a datelor scanate (pod describe, logs) in conversatie
 - o Incarcarea de fisiere (YAML, logs, configuratii) ca atasamente
 - o Selectarea nivelului de detaliu (describe, JSON, events, logs)
- Gestionarea istoricului conversatiilor
 - o Salvarea automata a tuturor conversatiilor in baza de date
 - o Reluarea conversatiilor anterioare
 - o Editarea/stergera/regenerarea titlului conversatiilor
- Autentificare si gestionarea conturilor de utilizator
 - o Inregistrare cu username, email si parola
 - o Login/Logout cu persistenta sesiunii

Introducere: constrangerile sistemului

- Clusterul Kubernetes trebuie sa fie accesibil din containerul backend (prin montarea fisierului kubeconfig si a binarului kubectl).
- Serverul AI necesita un model LLM (Ollama cu Llama 3.1) rulat local sau pe un server dedicat cu resurse GPU/CPU suficiente.
- Sistemul este deployat cu Docker Compose si necesita cel putin 4GB RAM total (2GB backend, 1GB frontend, 1GB+ pentru PostgreSQL si Ollama).
- Interfata web functioneaza in browser modern (Chrome, Firefox, Edge), fara instalare de software suplimentar pe client.
- Baza de date PostgreSQL stocheaza toate datele persistent; nu se doreste inlocuirea, ci integrarea cu sistemele existente.

Exemple de sarcini

Urmatoarele sarcini au fost colectate din observarea activitatilor inginerilor DevOps si a dezvoltatorilor care lucreaza cu Kubernetes, precum si din experienta proprie de administrare a clusterului licenta-cluster.

Sarcina 1: Diagnosticarea unui pod in CrashLoopBackOff

Andrei, inginer DevOps cu experienta de 3 ani, primeste o alerta ca serviciul backend-elearning din namespace-ul "elearning" nu mai raspunde. Deschide Kubexplain in browser si navigheaza la tab-ul "Pods Scanner". Introduce namespace-ul "elearning" si apasa "Scan for Pods". Vede ca podul backend-6f745b6fb5-jfsjv are statusul "CrashLoopBackOff" cu 15 restart-uri. Apasa pe pod pentru a vedea detaliile. In tab-ul "Logs" observa eroarea "java.lang.OutOfMemoryError: Java heap space". Selecteaza optiunea "Add context to chat" si se intoarce la chat. Scrie: "De ce crashuieste podul meu de backend?". AI-ul analizeaza logurile atasate si ii explica ca containerul depaseste limita de memorie si ii sugereaza sa creasca resources.limits.memory in deployment YAML.

Discutie: Aceasta este o sarcina frecventa si foarte importanta. Combina scanarea automata a clusterului cu analiza AI, reducand timpul de diagnosticare de la minute (manual cu kubectl) la secunde. Andrei este un utilizator tipic - experimentat dar care apreciaza automatizarea. Sarcina ilustreaza fluxul principal: Scan -> Inspect -> Add context -> Chat -> Solutie.

Sarcina 2: Investigarea unui pod blocat in Pending

Maria, administrator de sisteme, observa ca un nou deployment nu s-a finalizat. Podul postgres-backup-0 a ramas in starea "Pending" de 20 de minute. Deschide Kubexplain, scaneaza podurile din namespace-ul "databases" si identifica podul cu probleme. Apasa pe el, selecteaza tab-urile "Describe" si "Events", apoi adauga ambele ca context la chat. Intreaba: "De ce nu porneste acest pod?". AI-ul analizeaza evenimentele si identifica mesajul "0/3 nodes are available: 3 Insufficient memory". Ii explica ca niciun nod din cluster nu are suficienta memorie libera si ii sugereaza sa verifice consumul pe noduri sau sa adauge un nod nou.

Discutie: Sarcina demonstreaza o problema de scheduling frecvent intalnita. Maria utilizeaza multiple niveluri de detaliu (Describe + Events) pentru a oferi AI-ului context suficient. Sarcina este importanta si relativ frecventa, in special in clusterelor cu resurse limitate.

Sarcina 3: Atasarea unui fisier YAML pentru analiza

Radu, dezvoltator junior, a scris un manifest YAML pentru deployment-ul aplicatiei sale dar primeste erori la "kubectl apply". Nu intelege ce este gresit. Deschide Kubexplain, se logheaza cu contul sau, apoi apasa butonul "+" de langa caseta de text si selecteaza "Attach file". Incarca fisierul deployment.yaml de pe calculatorul sau. Scrie in chat: "Te rog analizeaza acest manifest YAML si spune-mi ce erori are". AI-ul parseaza fisierul atasat si identifica doua probleme: un indent gresit la spec.containers si lipsa campului "image" pentru al doilea container. Radu corecteaza fisierul si deploy-ul functioneaza.

Discutie: Sarcina demonstreaza functionalitatea de upload de fisiere si analiza statica a configuratiilor. Este o sarcina frecventa pentru dezvoltatorii juniori. Ilustreaza si faptul ca utilizatorul poate sa nu aiba acces direct la cluster - poate lucra doar cu fisiere locale.

Sarcina 4: Verificarea starii nodurilor clusterului

Elena, SRE (Site Reliability Engineer), face verificarea de dimineata a clusterului. Deschide Kubexplain si navigheaza la "Nodes Scanner". Apasa "Scan for Nodes" si vede 3 noduri: un master si doi workeri. Observa ca nodul worker-2 are statusul "NotReady". Selecteaza "Select Worker" pentru a bifa doar nodurile worker, apoi alege nivelul de detaliu "Describe" + "Events" si apasa "Add

context for 2 selected nodes". Contextul se adauga in chat. Intreaba: "Ce se intampla cu worker-2? De ce e NotReady?". AI-ul analizeaza output-ul de describe si identifica ca kubelet-ul nu mai comunica cu API server-ul de 15 minute. Sugereaza sa verifice daca nodul este accesibil prin SSH si daca serviciul kubelet functioneaza.

Discutie: Verificarea nodurilor este o sarcina de rutina efectuata zilnic. Scanner-ul de noduri ofera o vizualizare rapida a sanatatii intregului cluster. Selectarea in bulk a nodurilor si alegerea nivelului de detaliu demonstreaza eficienta interfetei pentru sarcini repetitive.

Sarcina 5: Reluarea unei conversatii anterioare

Andrei a investigat ieri o problema cu un serviciu si a primit o sugestie de la AI. Astazi vrea sa verifice daca solutia a functionat. Deschide Kubexplain, navigheaza la "Chat History" si gaseste conversatia cu titlul "Backend OOM Investigation". Apasa pe ea si conversatia anterioara se reincarca cu tot contextul. Scrie: "Am marit limita la 2Gi dar tot crashuieste. Ce altceva pot face?". AI-ul, avand contextul conversatiei anterioare, ii sugereaza sa verifice daca nu exista un memory leak in aplicatie si sa foloseasca un Java profiler.

Discutie: Aceasta sarcina demonstreaza persistenta conversatiilor si continuitatea contextului de-a lungul mai multor sesiuni. Este o functionalitate importanta care diferentiaza Kubexplain de un simplu chat AI - sistemul "isi aminteste" investigatiile anterioare. Sarcina este frecventa in cazul problemelor complexe care necesita mai multe iteratii de diagnosticare.

Sarcina 6: Adaugarea contextului scanat de poduri multiple

George, inginer DevOps, observa ca mai multe poduri din namespace-ul "production" au probleme simultan. Deschide Kubexplain, introduce namespace-ul "production" in Pods Scanner si apasa Scan. Vede 12 poduri, dintre care 4 au statusul diferit de "Running". Bifeaza "Select All" in bara de optiuni bulk, apoi deselecteaza manual podurile sanatoase. Alege nivelul de detaliu "Describe" + "Logs" si apasa "Add context for 4 selected pods". Intreaba in chat: "Am mai multe poduri cu probleme simultan. Care este cauza comuna?". AI-ul analizeaza cele 4 seturi de date si identifica un pattern comun: toate esueaza la conectarea la baza de date PostgreSQL. Sugereaza ca problema este la serviciul de baza de date, nu la podurile individuale.

Discutie: Aceasta sarcina complexa demonstreaza capacitatea sistemului de a corea informatii din surse multiple pentru a identifica o cauza radacina comuna. Selectarea in bulk si nivelurile de detaliu permit utilizatorului sa controleze cantitatea de context trimisa catre AI. Este o sarcina mai putin frecventa dar foarte importanta in situatii de outage.

Sarcina 7: Intrebare generala despre Kubernetes fara context de cluster

Ioana, studenta in anul 4, lucreaza la un proiect de semestru si nu intelege diferenta intre un Deployment si un StatefulSet in Kubernetes. Deschide Kubexplain si, fara sa scaneze vreun cluster, scrie direct in chat: "Explica-mi diferenta dintre Deployment si StatefulSet si cand sa folosesc fiecare". AI-ul ii ofera o explicatie detaliata, cu exemple de use-case-uri, folosind documentatia Kubernetes din baza de cunostinte (RAG). Ioana continua cu "Poti sa imi dai un exemplu de StatefulSet YAML pentru un cluster Redis?" si AI-ul genereaza un manifest complet.

Discutie: Aceasta sarcina demonstreaza ca Kubexplain nu este doar un instrument de diagnosticare, ci si un asistent educational. RAG-ul (Retrieval-Augmented Generation) permite raspunsuri bazate pe documentatia oficiala. Sarcina este frecventa pentru studentii si dezvoltatorii care invata Kubernetes.

Lista de cerinte

Ce trebuie inclus neaparat (Must Have):

- Chat conversational cu AI (LLM) pentru diagnosticarea problemelor Kubernetes
- Scanarea podurilor dintr-un namespace specificat cu afisarea statusului
- Scanarea nodurilor clusterului cu afisarea rolului si statusului
- Vizualizare detaliata a resurselor: Describe, JSON, Events, Logs
- Adaugarea datelor scanate ca context in conversatia de chat
- Persistenta conversatiilor in baza de date PostgreSQL
- Sistem de autentificare (Register/Login)
- Protocol propriu de comunicare (kdiag/1.0) intre backend si serverul AI
- Integrare cu un LLM local (Ollama cu Llama 3.1)

Ce trebuie inclus (Should Have):

- Istoric complet al conversatiilor cu posibilitatea de reluare
- Upload de fisiere (YAML, logs, configuratii) ca atasamente la conversatie
- RAG (Retrieval-Augmented Generation) cu documentatie Kubernetes scraping
- Dynamic RAG - cautare automata pe kubernetes.io cand LLM-ul nu gaseste raspuns
- Generare automata de titlu pentru conversatii (folosind LLM)
- Selectare in bulk a podurilor/nodurilor cu niveluri de detaliu configurabile
- Fallback heuristic cand serverul AI nu este disponibil
- Sistem de feedback (rating) pentru raspunsurile AI

Ce ar putea fi inclus (Could Have):

- Dashboard cu metrici de performanta a clusterului in timp real
- Notificari automate cand un pod intra in stare de eroare
- Suport pentru mai multe modele LLM (selectie din Settings)
- Export conversatii ca PDF sau Markdown
- Integrare cu sisteme de alertare existente (Prometheus, Grafana)
- Dark/Light mode toggle complet functional
- Autocomplete pentru namespace-uri si nume de resurse

Exclus:

- Executarea automata de comenzi kubectl de remediere fara confirmare umana
- Administrarea directa a clusterului (creare/stergere resurse) prin interfata

- Suport multi-cluster simultan in aceeași sesiune
- Analiza de securitate sau audit RBAC

Justificare: Executarea automată de comenzi de remediere a fost exclusă deliberat din motive de siguranță - un sistem AI nu trebuie să facă modificări neautorizate într-un cluster de producție. Administrarea directă depășește scopul unui instrument de diagnosticare. Multi-cluster-ul și auditul de securitate ar crește complexitatea semnificativ fără a aduce valoare pentru MVP.

5. Sectiunea 2: Prototip si parcurgere orientata pe sarcini

Prototipul Kubexplain este o aplicatie web functionala, implementata cu urmatorul stack tehnologic:

- Frontend: HTML5 + CSS3 + JavaScript vanilla, servit prin Express.js (Node.js)
- Backend: Spring Boot (Java 17) cu API REST
- Server AI: Spring Boot (Java 17) cu integrare Ollama (Llama 3.1)
- Baza de date: PostgreSQL 16
- Containerizare: Docker Compose cu 3 servicii + baza de date
- Protocol: kdiag/1.0 - protocol propriu pentru comunicarea backend-AI server

Interfata principala este de tip single-page application cu sidebar de navigare si cinci tab-uri: Home (Chat), Chat History, Pods Scanner, Nodes Scanner si Settings.

Parcurearea sarcinii 1: Diagnosticarea unui pod in CrashLoopBackOff

Subsarcina 1: Autentificarea in sistem

- a) Utilizatorul apasa butonul "Login" din sidebar-ul stang. Se deschide un modal cu formularul de autentificare.
- b) Introduce email-ul si parola. Apasa "Login".
- c) Daca credentialele sunt corecte, modalul se inchide, butonul "Login" se transforma in "Logout" si apare username-ul utilizatorului.

Subsarcina 2: Scanarea podurilor

- a) Se navigheaza la tab-ul "Pods Scanner" din sidebar.
- b) In campul "Namespace" se introduce "elearning".
- c) Se apasa butonul "Scan for Pods".
- d) Sistemul executa "kubectl get pods -n elearning -o json" pe server si afiseaza lista podurilor cu informatii: nume, status, restart-uri, varsta.
- e) Podurile cu probleme sunt evidentiata vizual (culori diferite pentru statusuri: verde pentru Running, rosu pentru CrashLoopBackOff, galben pentru Pending).

Subsarcina 3: Inspectia detaliata a podului

- a) Se apasa pe podul backend-6f745b6fb5-jfsjv cu status CrashLoopBackOff.
- b) Se deschide un modal cu detalii complete, organizate in tab-uri: Describe, JSON, Events, Logs.
- c) In tab-ul Logs, se observa eroarea OOM.
- d) Se apasa "Add context to chat" - datele selectate se adauga automat ca artefact in conversatia curenta.

Subsarcina 4: Conversatia cu AI-ul

- a) Se revine la tab-ul Home (chat). In zona de preview a atasamentelor se vede contextul adaugat.
- b) Se scrie intrebarea: "De ce crashuieste podul meu de backend?".
- c) Se apasa Send (sau Enter). Mesajul + atasamentele sunt trimise la backend (port 8080), care le forwardaaza catre AI Server (port 8090) folosind protocolul kdiag/1.0.
- d) AI Server construiesc un prompt de sistem cu documentatia Kubernetes relevanta (RAG), adauga mesajul utilizatorului cu

e) Raspunsul AI-ului apare in zona de mesaje, formatat cu Markdown: paragrafe boldate pentru cauza, sugestii numerotate pentru solutii.

Parcursarea sarcinii 2: Reluarea unei conversatii si adaugarea de fisier

Subsarcina 1: Accesarea istoricului

- a) Se navigheaza la tab-ul "Chat History".
- b) Se afiseaza lista conversatiilor anterioare cu titlu, data si preview.
- c) Se apasa pe conversatia dorita. Mesajele anterioare se reincarca in zona de chat.

Subsarcina 2: Atasarea unui fisier

- a) Se apasa butonul "+" de langa campul de text.
- b) Se selecteaza "Attach file" din dropdown.
- c) Se deschide selectorul de fisiere. Se alege deployment.yaml.
- d) Fisierul apare in zona de preview a atasamentelor cu numele, tipul MIME si dimensiunea.
- e) Se scrie intrebarea si se trimite. Fisierul este serializat si trimis ca artefact in payload-ul kdiag/1.0.

Subsarcina 3: Gestionarea conversatiei

- a) Titlul conversatiei este generat automat de LLM dupa primul mesaj.
 - b) Utilizatorul poate edita titlul apasand butonul de editare.
 - c) Se poate sterge conversatia din istoric cu confirmare.
 - d) Se poate regenera titlul automat.
-

Preocupari majore din parcursarea sarcinilor de indeplinit

Fluxul Scan -> Context -> Chat necesita mai putini pasi

- In prezent, utilizatorul trebuie sa: (1) navigheze la Pods Scanner, (2) scaneze, (3) apese pe pod, (4) apese "Add context", (5) revina la chat, (6) scrie intrebarea. Fluxul ar putea fi simplificat prin adaugarea unui buton "Quick Debug" direct pe fiecare pod din lista.

Cantitatea de context poate depasi limita LLM-ului

- Cand se adauga context pentru 4+ poduri cu Describe + Logs, payload-ul poate depasi context window-ul LLM-ului. Sistemul trunchiaza automat la 10.000 caractere per artefact, dar utilizatorul nu este avertizat. Se sugereaza adaugarea unui indicator vizual al dimensiunii contextului.

Latenta raspunsurilor AI

- Cu Llama 3.1 rulat local pe CPU, raspunsurile pot dura 30-60 secunde. In prototipul actual, utilizatorul vede doar un spinner fara progres. Se sugereaza implementarea streaming-ului de raspunsuri (token by token) pentru o experienta mai responsiva.

Feedback-ul si calitatea raspunsurilor

- Sistemul de feedback (rating) permite colectarea aprecierilor, dar nu exista inca un mecanism de a folosi aceste date pentru imbunatatirea raspunsurilor. Se sugereaza implementarea unui sistem de RAG care sa salveze rezolvarile validate si sa le refoloseasca in cazuri similare.

Se sugereaza redesign minor al interfetei pentru:

- Integrarea mai fluida a scanner-elor in fluxul de chat
- Afisarea starii clusterului intr-un dashboard compact pe pagina principala
- Indicatori vizuali pentru loading/streaming al raspunsurilor AI

6. Bibliografie

1. Kubernetes Documentation. [Online] <https://kubernetes.io/docs/>
2. Ollama - Run Large Language Models Locally. [Online] <https://ollama.com/>
3. Spring Boot Reference Documentation. [Online] <https://spring.io/projects/spring-boot>
4. Express.js Documentation. [Online] <https://expressjs.com/>
5. PostgreSQL Documentation. [Online] <https://www.postgresql.org/docs/>
6. Docker Compose Documentation. [Online] <https://docs.docker.com/compose/>
7. Norman, Donald A. "The Design of Everyday Things." Basic Books, 2013.
8. Greenberg, Saul. CPSC 481 Human Computer Interaction I: Principles and Design. University of Calgary. [Online] <http://pages.cpsc.ucalgary.ca/~saul/481/>
9. Lewis, C. & Rieman, J. Task-centered User Interface Design. 1993.
10. Meta AI. "Llama 3.1 - Open Foundation and Fine-Tuned Chat Models." 2024.